

Examen2324_Resuelto.pdf



Anónimo



Algorítmica



2º Grado en Ingeniería Informática



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada

Formamos
talento para un futuro
Sostenible



MÁSTER EN

**Big Data &
Business Analytics**

EOI Escuela de
organización
industrial

[saber más](#)

SÁCATE EL CARNET ESTE VERANO ¡SIN LISTA DE ESPERA!

15% DE DESCUENTO
CON EL CÓDIGO **HVWU024**



EMPIEZA
PRÁCTICAS
Y SUBE
A EXAMEN
CUANDO
QUIERAS



hoy-voy
Valladolid
P. Isabel
la Católica,
28



UNIVERSIDAD
DE GRANADA

Departamento de Ciencias de la
Computación e Inteligencia Artificial

Algorítmica (2023/24)
2º Grado en Ingeniería Informática
Examen ejemplo

Normas para la realización del examen:

Duración: 2.5 horas

- El único material permitido durante la realización del examen es un bolígrafo azul o negro.
- Debe disponer de un documento oficial que acredite su identidad a disposición del profesor.
- No olvide escribir su nombre completo y grupo en todos y cada uno de los folios que entregue.

◁ Ejercicio 1 ▷ ECUACIÓN RECURRENTE DE EFICIENCIA

[2 puntos]

- Calcula la eficiencia del algoritmo cuyo tiempo de ejecución viene dado por la siguiente ecuación en recurrencias: $T(n) = 3 \cdot T(n/3) + 2 \cdot n$ (1 punto).

$T(n) = 3 \cdot T(n/3) + 2 \cdot n$, hacemos cambio de variable $n = 3^m$, con que nos da $T(3^m) = 3 \cdot T(3^{m-1}) + 2 \cdot 3^m$. La expresión sería $T(3^m) - 3 \cdot T(3^{m-1}) = 2 \cdot 3^m$, con lo que es una expresión no homogénea de la forma $x - 3 = 2 \cdot 3^m = q(n) \cdot b^m$, en donde $b=3$, $q(n)=2 \Rightarrow d=0$.

La ecuación característica la resolvemos primero sin parte no homogénea: $(x-3)$. Al añadir la parte no homogénea se debe de añadir $(x-b)^{d+1}$. $(x-3) \cdot (x-3) = (x-3)^2$. Por tanto, la ecuación de eficiencia es $O(c_1 \cdot 3^m + c_2 \cdot m \cdot 3^m) = O(c_1 \cdot n + c_2 \cdot \log_3(n) \cdot n) = O(n + \log_3(n) \cdot n) = O(\log_3(n) \cdot n)$

- Tenemos una clase **Jugador** que almacena una lista de cartas, usando una implementación de memoria, en donde existen un tamaño actual (`num_cartas`), y una memoria reservada (`num_reserva_memoria`), que puede ser igual o mayor que el número de cartas. Dado el siguiente código, indicar la eficiencia en el mejor caso y peor caso, en donde N es el actual número actual de cartas. (1 punto)

```
1      bool Jugador::coge_carta(const Carta &c) {
2          Carta *p;
3
4          if (num_cartas == num_reserva_memoria) {
5              num_reserva_memoria += 1;
6              p = new Carta[num_cartas+1];
7              if (p == nullptr) {
8                  return false;
9              }
10
11             for (int i = 0; i < num_cartas; i++) {
12                 p[i] = cartas[i];
13             }
14
15             delete[] cartas;
16             cartas = p;
17         }
18
19         cartas[num_cartas] = c;
20         num_cartas += 1;
21         return true;
22     }
```

Eficiencia `coge_carta(n)`=Eficiencia(If=4-17)+Eficiencia(Bloque2=19-21)

$O(4 - 17) = \Omega(4 - 17) = O(4 - 9) + O(11 - 13) + O(15 - 16) = O(1) + O(1 + n \cdot 1) + O(1) = O(n)$
 $O(\text{ParteElse}) = \Omega(\text{ParteElse}) = O(\text{if}) = O(\text{Condicion} + \text{MaxO}(\text{ParteIf}), O(\text{ParteElse})) = O(n) \Omega(\text{if}) = \Omega(\text{Conficion} + \text{Min}\Omega(\text{ParteIf}), \Omega(\text{ParteElse})) = O(1 + 1) = O(1)$

En el mejor caso entra en el if, siempre tiene memoria, y sería la eficiencia $O(1)$.

En el peor caso entra en el primer if, no tiene memoria, y sería la eficiencia $O(n)$ (por tener que copiar los valores anteriores en el nuevo array).

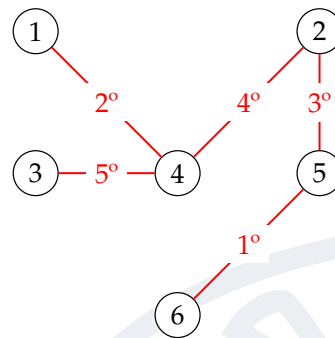
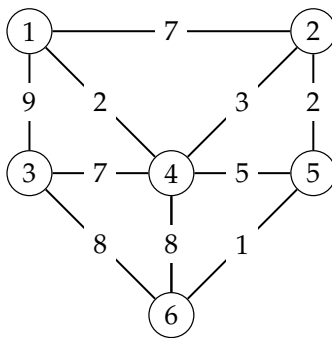
◁ Ejercicio 2 ▷ OPTIMIZANDO GRAFO

[1.5 puntos]

- Dado el siguiente grafo con pesos, obtener el subgrafo con peso minimal con alguno de los algoritmos vistos en clase, indicando cuál y explicándolo paso a paso. ¿La solución obtenida es óptima? ¿Podría haber alguna otra óptima? (0.5 puntos)

Para hacerlo aplico Kruskal, que escoge las aristas de menor a mayor peso, descartando aquellas que generasen un ciclo. Por tanto, las aristas serían en ese orden $\{5, 6\}$, $\{1, 4\}$, $\{2, 5\}$, $\{2, 4\}$, $\{3, 4\}$. El resto de aristas se ignoran por generar ciclos.

El coste total es $1+2+3+2+7 = 15$.



La solución obtenida es óptima, ya que este algoritmo (y la otra opción, Prim) generan algoritmos óptimos. En general puede haber más de una solución óptima, con el mismo coste. Sin embargo, en este ejemplo no se da el caso.

- Calcular el camino más corto entre el nodo 1 y 5, ¿qué algoritmo lo calcula? Indicar el proceso paso a paso. (1 punto)

Para calcular el camino más corto entre dos nodos, escogemos el algoritmo Dijkstra visto en clase. Este algoritmo parte de un subgrafo que va aumentando añadiendo nodos hasta que está todos en él. En cada paso escoge siempre la arista con menor peso que parta de un nodo existente en el subgrafo a otro que todavía no esté.

Paso	Desde	1	2	3	4	5	6
1º	1	-	7	9	2	α	α
2º	4	-	5	9	-	7	10
3º	2	-	-	9	-	7	10
4º	5	-	-	9	-	-	8
3º	6	-	-	9	-	-	-

Indico en la tabla anterior los distintos pasos.

Paso 1 el nodo más cercano es 4, así que se añade al subgrafo.

Paso 2 Desde el nodo 4, se puede llegar al nodo 2 con un peso menor a ir directamente ($2+3$ vs 7), así que se actualiza en la lista. También se actualizan las distancias a los nuevos nodos (5 y 6). Se escoge el nodo 2.

Paso 3 Desde el nodo dos se puede acceder a 5 , con el mismo coste.

Paso 4 Se escoge el nodo 5 para seguir actualizando los tiempos. El resto de los pasos no nos interesan, porque aunque Dijkstra calcula el camino más corto de un nodo a todos los demás, sólo nos interesa el camino hasta el nodo 5 , obtenido en este paso.

...

1. EXISTEN TRES TIPOS DE ESCALA PARA MEDIR LA TEMPERATURA. ¿CUÁLES SON?

- A) CELSIUS, FAHRENHEIT Y KELVIN**
- B) DORITOS FLAMIN' HOT, RUFFLES FLAMIN' HOT Y CHEETOS FLAMIN' HOT**

2. SI PABLO TIENE 9 DORITOS FLAMIN' HOT Y MARÍA LE QUITA UNO, ¿QUIÉN SE PICA MÁS?

- A) PABLO**
- B) LA LENGUA DE MARÍA**

3. UN TREN SALE DE MADRID A 200 KM/H A LAS 9 AM, Y OTRO DE BARCELONA A 100 KM/H A LAS 10 AM... Y TÚ, ¿ESTÁS FLAMIN' HOT?

- A) EL DE BARCELONA**
- B) SÍ, ESTOY FLAMIN' HOT**

SI HAS RESPONDIDO B) A TODAS LAS PREGUNTAS, ENHORABUENA, ESTÁS FLAMIN' HOT

SI HAS RESPONDIDO A) EN CUALQUIERA DE ELLAS, VUELVE AL INICIO DEL TEST



Algorítmica



Comparte estos flyers en tu clase y consigue más dinero y recompensas



Banco de apuntes de la

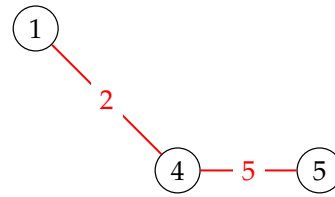
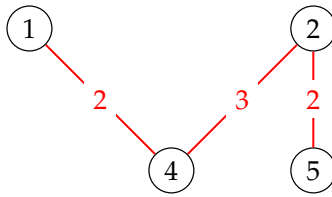
WUOLAH

- 1** Imprime esta hoja
- 2** Recorta por la mitad
- 3** Coloca en un lugar visible para que tus compis puedan escanear y acceder a apuntes

- 4** Llévate dinero por cada descarga de los documentos descargados a través de tu QR

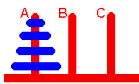


Hay dos opciones, ambas con el mismo camino óptimo de 7:



◁ Ejercicio 3 ▷ RESOLVIENDO UN PROBLEMA CON DIVIDE Y VENCERÁS

[1 punto]



Se desea resolver el problema de las torres de Hanoi, que consiste en que hay tres torres, y se desea mover una serie de discos de la torre A a la torre C (la torre B puede usarse de forma intermedia), tal y como muestra el gráfico.

Para ello, la forma más sencilla es por medio de la técnica divide y vencerás. Se pide implementar la función

```
void hanoi(char From, char To, char Other, int n)
```

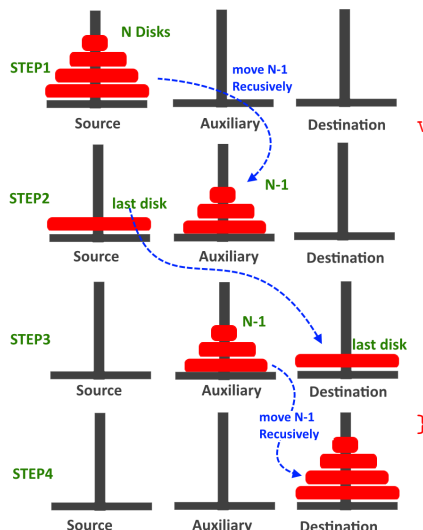
En donde *From*, y *To* son los identificadores de la columna origen y destino, y *n* es el número de discos a mover. La función no devuelve nada, sólo escribe por pantalla los movimientos para que el humano sea quien lo realice. Por ejemplo, dado:

```
hanoi('A', 'B', 'C', n);
```

Debería de devolver los mensajes como:

```
Muevo disco de 'A' a 'C'
Muevo disco de 'A' a 'B'
...
```

Este es un ejemplo típico en el que la estrategia divide y vencerás es muy sencilla.



```
void hanoi(char From, char To, char Other, int n) {
    if (n == 1) {
        cout << "Muevo disco de " << From << " a " << To << endl;
    }
    else {
        hanoi(From, Other, To, n-1);
        cout << "Muevo disco de " << From << " a " << To << endl;
        hanoi(Other, To, From, n-1);
    }
}
```

◁ Ejercicio 4 ▷ ALGORITMOS VORACES/GREEDY

[1.5 puntos]

- Tenemos un juego con varios jugadores en el que tenemos una serie de monedas distintas en fila. Cada jugador puede escoger la moneda de un extremo, izquierdo o derecha, y luego le toca el turno al siguiente.

SÁCATE EL CARNET ESTE VERANO ¡SIN LISTA DE ESPERA!

15% DE DESCUENTO
CON EL CÓDIGO **HVWU024**



EMPIEZA
PRÁCTICAS
Y SUBE
A EXAMEN
CUANDO
QUIERAS



hoy-voy
Valladolid
P. Isabel
la Católica,
28

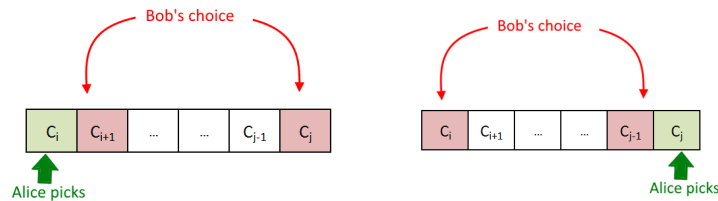


UNIVERSIDAD
DE GRANADA

Departamento de Ciencias de la
Computación e Inteligencia Artificial

Algorítmica (2023/24)
2º Grado en Ingeniería Informática
Examen ejemplo

Cada jugador tiene una puntuación igual a la suma del valor de las monedas, y quien tenga más monedas gana.



- Diseñar un algoritmo voraz/greedy que permita obtener una adecuada asignación de movimientos para maximizar, indicando sus componentes. **(1 punto)**

Para diseñar el algoritmo greedy, es necesario definir los siguientes componentes:

Función Objetivo es maximizar el valor de las monedas obtenidas.

Lista de Candidatos son las opciones en cada iteración, es decir, la moneda inicial C_i y final C_j , de entre las que tenemos que escoger.

Lista de candidatos ya usados son las monedas cogidas.

Criterio de Factibilidad que sean las monedas primeras y/o última.

Función Solución Cuando ya no queden monedas para escoger.

Función de Selección Escoger de entre las monedas C_i y C_j aquella con mayor valor, por tanto la heurística sería $Heurística = \text{Max}\{C_i, C_j\}$.

- Suponiendo que sólo hay dos jugadores, ¿se podría mejorar la heurística? En ese caso, proponerlo. Razona la respuesta. **(0.5 puntos)**

Sí se podría mejorar, en vez de hacer $Heurística(Mov) = \text{Max}\{C_i, C_j\}$, se podría valorar el valor que le dejamos al contrario, como $Heurística2 = \text{Max}\{C_i - C_{i+1}, C_j - C_{j-1}\}$.

◁ Ejercicio 5 ▷ PROGRAMACIÓN DINÁMICA

[2 puntos]

Se tiene una cuerda de tamaño N que queremos vender, y en la tienda para cada longitud posible (entre 1 y N) existe un precio distinto para comprar. Un posible ejemplo de precio sería:

Longitud	1	2	3	4	5	6	7	8
Precio	1	5	8	9	10	17	17	20

Viendo la lista observamos que el tendero no es muy listo, y es posible obtener mejor resultado dividiéndola en partes, y vendiendo cada una. Por ejemplo, en el ejemplo anterior si tenemos cuerda de longitud 8, en vez de venderla directamente (nos darían 20), podemos cortarla en dos trozos, de 6 y de 2, y obtendríamos 22.

Vamos a diseñar una función, que dada una longitud nos diga el mejor precio posible.

- Plantée el diseño usando Programación Dinámica para resolver el problema. **(1 punto)**.

Es un problema claro de optimización. Para resolverlo usamos un vector de mejor valores para cada longitud MV .

Inicialización: $MV[i] = \text{precios}[i] \forall i \in \{1, \dots, \text{tam}\}$

Expresión recursiva: $MV[i] = \max_{j=1, j < i} \{MV[j], MV[i - j]\}$.

- Implemente la función para resolver el problema: **(1 punto)**

`int precioMaximo(int precios[], int tam, int longitud, list<int> &partes);`

WUOLAH

En donde *partes* sería la lista de longitudes en las que partirla para conseguir el precio devuelto por parámetro.

Se puede implementar sin escribir en *partes*, con una calificación de **(0.5 puntos)**.

Primero la versión sin partes, que es más sencillo:

```
1 int precioMaximo(int *precios, int tam, int longitud) {
2     int *acumulados;
3
4     // Copio en la posición i el precio de i, para evitar problemas con índices
5     acumulados = new int[tam+1];
6
7     // Inicio con los precios
8     for (int i = 1; i <= tam; i++) {
9         acumulados[i] = precios[i-1];
10    }
11
12    for (int i = 2; i <= longitud; i++) {
13        // Puede ser hasta j <= i/2, pero con i es correcto
14        for (int j = 1; j < i; j++) {
15            int value = acumulados[j] + acumulados[i-j];
16
17            // Si dividido en dos se mejora el precio lo sustituyo
18            if (value > acumulados[i]) {
19                acumulados[i] = value;
20            }
21        }
22    }
23
24    return acumulados[longitud];
25 }
```

Se puede ver que es relativamente sencillo. En el código se compara con j desde 1 hasta i, aunque realmente bastaría entre 1 y i/2, ya que si no se comparan varias veces las mismas combinaciones.

La opción usando el vector de partes es mediante una tabla hash de valores, guardando para cada longitud el conjunto de valores con los que se puede obtener el mejor valor (por defecto, es directamente con la longitud). Cuando se obtiene una mejora combinando dos valores, se reemplazan los componentes de la combinación anterior, y se añaden los componentes de los valores combinados.

`append_solution(origen, destino)` es una función que añade los elementos de la lista *origen* a la lista *destino*.

```
1 int precioMaximo(int *precios, int tam, int longitud, list<int> &partes) {
2     int *acumulados;
3     map<int, list<int> > solutions;
4
5     // Copio en la posición i el precio de i, para evitar problemas con índices
6     acumulados = new int[tam+1];
7     acumulados[0] = 0;
8
9     // Inicio con los precios
10    for (int i = 1; i <= tam; i++) {
11        acumulados[i] = precios[i-1];
12        solutions[i].push_back(i);
13    }
14
15    for (int i = 2; i <= longitud; i++) {
16        // Puede ser hasta j <= i/2, pero con i es correcto
```



```
17     for (int j = 1; j < i; j++) {
18         int value = acumulados[j] + acumulados[i-j];
19
20         // Si dividido en dos se mejora el precio lo sustituto
21         if (value > acumulados[i]) {
22             solutions[i].clear();
23             append_list(solutions[j], solutions[i]);
24             append_list(solutions[i-j], solutions[i]);
25             acumulados[i] = value;
26         }
27     }
28 }
29
30 append_list(solutions[longitud], partes);
31 return acumulados[longitud];
32 }
```

< Ejercicio 6 > EXPLORACIÓN DE GRAFO**[2 puntos]**

Se desea un programa que divida un vector de N números enteros (positivos o negativos) en dos grupos, en donde cada grupo sume lo mismo. Es decir, que dado el vector {5, 3, 4, 2} lo divida en los grupos {5, 2} y {3, 4}. Para ello, se busca una función que dado el vector de N enteros, escriba en un vector de N valores un valor 0 ó 1 indicando si dicho número pertenece al primer grupo o al segundo. Es decir, en el ejemplo anterior devolvería el vector grupos los valores {0, 1, 1, 0}. Si no puede devolverlo deberá de devolver la función *false*, y *true* en caso contrario.

La cabecera deseada es:

```
bool divide(int *values, int *grupos, int tam);
```

Para implementarlo, hay que implementar un método recursivo (con mayor número de parámetro) que permita resolverlo mediante la técnica de *Backtracking*.

Indicar el pseudocódigo de la función siguiente resuelta mediante *backtracking* (2 puntos).

Pista: valorar pasar por parámetro la diferencia de valor entre los grupos.

```
1 bool divide(int *values, int *group, int begin, int end, int diferencia) {
2     if (begin > end) {
3         return (diferencia == 0);
4     }
5
6     for(int i = 0; i < 2; i++) {
7         group[begin] = i;
8         int val = (i == 0) ? values[begin] : -values[begin];
9
10        if (divide(values, group, begin+1, end, diferencia+val)) {
11            return true;
12        }
13        else {
14            group[begin] = -i;
15        }
16    }
17
18    return false;
19 }
20
21 bool divide(int *values, int *group, int tam) {
22     return divide(values, group, 0, tam-1, 0);
23 }
```